

AI WeatherWise

1.Introduction:

Project Title: AI Weatherwise

Team Members:

S. Sanjay (*Team Lead / Project Manager*) :

Administrator & Coordinator Manages

C. Manigandan (*Backend Engineer (Lead)*) :

Node.js / Express Developer

R. Surendhar (*Database Engineer*):

MongoDB / Mongoose Architect

S. Sathish (*AI Systems Integration Engineer*):

AI & Service Developer

M. Sathesh (*QA / API Testing Specialist*) :

Quality Assurance & Security Engineer

2.Project Overview:

AI WeatherWise is a RESTful backend platform developed with Node.js, Express.js, MongoDB, and Mongoose ODM. It integrates real-time weather metrics with Google Gemini AI (@google/genai / @google/generative-ai) to deliver natural language weather summaries and context-aware activity recommendations based on live environmental conditions.

The system features JSON Web Token (JWT) authentication, bcrypt password hashing, input sanitization, centralized error handling, resilient fallback modes for API keys, and full CRUD capability for managing favorite locations.

3.Problem Statement & Scenario-Based Case Study

Background & Scenario:

Alex is an avid traveler who frequently visits various cities. He needs a streamlined way to track real-time weather metrics across multiple locations and receive intelligent, natural-language activity and clothing suggestions rather than sifting through raw numbers.

Core Challenges:

Difficulty efficiently tracking and managing multiple favorite weather locations.

Absence of personalized, context-aware suggestions for outdoor activities or clothing based on live weather conditions.

Overreliance on raw metrics (temperature, humidity, pressure) instead of actionable, natural language insights.

Lack of a secure, cross-device synced system to store user preferences and saved locations.

Proposed Solution:

AI WeatherWise acts as a centralized orchestrator between user location preferences, live weather fetching, and Google Gemini AI insights. It manages user accounts securely via JWT and provides tailored, weather-driven workflows.

4.System Requirements

Software Requirements

Operating System: Windows 10/11, macOS, or Linux (Cross-platform)

Runtime Environment: Node.js (v16 or higher)

Package Manager: npm (v8 or higher)

Database: MongoDB (Local or MongoDB Atlas)

AI Integration: @google/genai / @google/generative-ai SDK

API Testing Tools: Postman or Thunder Client

Code Editor: Visual Studio Code

Hardware Requirements

Processor: Intel Core i5 (8th Gen+) / AMD Ryzen 5 or higher

RAM: Minimum 8 GB (16 GB recommended for running concurrent server and database instances)

Storage Space: Minimum 1 GB available disk space

5. System Architecture & Design Patterns

Model-View-Controller (MVC) Pattern

The backend adheres strictly to the MVC Architecture to separate concerns and enhance code maintainability:

Model Layer (Data Layer): Defines strongly enforced Mongoose schemas (User, Location) for MongoDB. Handles data validation, indices, and defaults.

Controller Layer (Business Logic): Captures requests, validates request payloads, handles business workflows (fetching weather via OpenWeatherMap, generating AI recommendations via Gemini), and packages responses into standard JSON.

View Layer (API Router): Functions as a headless backend API routing layer. Maps HTTP request verbs and paths to their corresponding controller functions

6. Database Architecture & ER Design

Entity Definitions

1. User Entity

`_id`: ObjectId (Primary Key, Auto-generated)

`name`: String (Required)

`email`: String (Required, Unique)

`password`: String (Required, Hashed via bcrypt)

`role`: String (Default: "reader", Options: "reader", "admin")

2. Location Entity

_id: ObjectId (Primary Key, Auto-generated)

user: ObjectId (Foreign Key referencing User collection)

city: String (Required)

country: String (Required)

createdAt: Date (Default: Date.now)

Relationship Mapping

User \rightarrow Location (One-to-Many): One registered user can save and track multiple favorite locations.

7.Roles & Responsibilities

Admin Role: Oversees global user accounts, monitors system health, database integrity, and monitors API metrics.

Registered User: Manages profile authentication, creates and deletes favorite locations, retrieves live weather data, and triggers AI-driven activity recommendations.

Anonymous Public User: Accesses public weather routes for quick weather checks without logging in.